

18. Reciprocal Rank Fusion (RRF) for Hybrid Search

Phase: L **Code:** `search_api/services/rrf.py` , `search_api/services/search.py`

What it is

Reciprocal Rank Fusion (RRF) is a technique for combining multiple ranked lists into a single final ranking. In our system, we have two completely different ways of finding relevant results:

1. **Vector search** -- compares the meaning of the query against stored embeddings using cosine distance. Returns a similarity score between 0 and 1.
2. **BM25 search** -- matches keywords (lemmatized tokens) against a full-text index. Returns a relevance score that can range from 0 to any positive number.

The problem: these two score types are incomparable. A vector cosine similarity of 0.85 and a BM25 `ts_rank` of 3.7 are on completely different scales. You cannot meaningfully do `0.7 * 0.85 + 0.3 * 3.7` -- whichever source produces larger raw numbers would dominate the combined score regardless of actual relevance.

RRF solves this by ignoring the raw scores entirely. It only cares about the rank position (1st, 2nd, 3rd...) of each item in each list. The formula is:

```
score(item) = sum over each source of: weight / (k + rank_in_that_source)
```

Where `k` is a constant (default 60) that dampens the difference between adjacent ranks.

Real-life analogy

Imagine you are choosing a restaurant for dinner. You ask two friends for recommendations:

- **Friend A (the foodie)** ranks restaurants by taste on a scale of 1-100.

- **Friend B (the budget expert)** ranks them by value-for-money as a dollar amount saved per visit.

Friend A says: "Restaurant X scores 92, Restaurant Y scores 88." Friend B says: "Restaurant X saves you \$12, Restaurant Z saves you \$45."

You cannot meaningfully average 92 and \$12 -- the units are incomparable. But what you can do is look at their rankings:

- Friend A's ranking: #1 X, #2 Y
- Friend B's ranking: #1 Z, #2 X

Now give each restaurant a score based on its position: $1/(60+\text{rank})$. Restaurant X appears in both lists (#1 for A, #2 for B), so it gets $1/61 + 1/62 = 0.0326$. Restaurant Y only appears in A's list at #2, so it gets $1/62 = 0.0161$. Restaurant Z only appears in B's list at #1, so it gets $1/61 = 0.0164$.

Final ranking: X (0.0326), Z (0.0164), Y (0.0161). Restaurant X wins because it appeared high in both lists -- a signal that it is genuinely good across multiple criteria, not just one.

The $k=60$ constant is like saying "I don't want to overreact to small differences in position." Without it ($k=0$), the #1 result would get a score of 1.0 and #2 would get 0.5 -- a 2x gap for a single position difference. With $k=60$, #1 gets $1/61$ and #2 gets $1/62$ -- a much gentler slope that lets items appearing in multiple lists accumulate score.

Why we used it here

Two requirements drive the RRF choice:

R13 (Hybrid retrieval) requires that both vector and BM25 results are combined into a single ranking. The system must benefit from the strengths of both approaches -- vector search captures semantic meaning (finding "pumps" when the user types "water pressure equipment"), while BM25 captures exact keyword matches (finding "Bosch" when the user types "Bosch").

R14 (Bilingual RRF weighting) explicitly requires RRF fusion. The CLAUDE.md and SPEC.md forbid weighted-average blending ($0.7*\text{vector} + 0.3*\text{bm25}$) because the scores are on incomparable scales. RRF is the only approved fusion method.

The $k=60$ default comes from the original RRF paper (Cormack, Clarke & Buettcher, 2009). It was empirically shown to work well across many retrieval

tasks. A smaller k (like 1) would heavily favor top-ranked items; a larger k (like 1000) would flatten all ranks to nearly equal scores. 60 is the standard starting point, and we have not needed to tune it.

Code walkthrough

1. The RRF fusion function -- `search_api/services/rrf.py`

```
DEFAULT_K = 60                # standard RRF constant from the original paper
SECONDARY_LANGUAGE_WEIGHT = 0.5 # cross-language BM25 results get half weight

def fuse_rankings(
    vector_results: list[str],          # IDs sorted by vector similarity (best first)
    bm25_primary_results: list[str],    # IDs sorted by BM25 ts_rank (best first)
    bm25_secondary_results: list[str] | None = None, # cross-language fallback IDs
    k: int = DEFAULT_K,
) -> list[tuple[str, float]]:
    bm25_secondary_results = bm25_secondary_results or []
    scores: dict[str, float] = {}      # accumulates each item's total RRF score

    # Source 1: vector search -- full weight (1.0)
    # rank starts at 1 (not 0), so the best vector match gets 1/(60+1)
    for rank, item_id in enumerate(vector_results, start=1):
        scores[item_id] = scores.get(item_id, 0.0) + 1 / (k + rank)

    # Source 2: BM25 primary language -- full weight (1.0)
    # same formula, same k -- the only thing that differs is the input list
    for rank, item_id in enumerate(bm25_primary_results, start=1):
        scores[item_id] = scores.get(item_id, 0.0) + 1 / (k + rank)

    # Source 3: BM25 secondary language -- half weight (0.5)
    # multiplied by 0.5 so cross-language matches don't outrank same-language ones
    for rank, item_id in enumerate(bm25_secondary_results, start=1):
        scores[item_id] = scores.get(item_id, 0.0) + SECONDARY_LANGUAGE_WEIGHT / (k + rank)

    # Sort by accumulated score, highest first
    fused = sorted(scores.items(), key=lambda pair: pair[1], reverse=True)
    return fused    # list of (item_id, rrf_score) tuples
```

Key design decisions:

- **Items missing from a source contribute nothing.** If an item only appears in the vector list but not in BM25, it only gets a score from the vector source. It is not treated as "ranked last" -- that would unfairly penalize items that one method legitimately did not find.
- **Secondary language gets 0.5 weight.** This means a #1 secondary-language BM25 result ($0.5/61 = 0.0082$) scores lower than a #1 primary-

language BM25 result ($1/61 = 0.0164$). It can still surface if it also appears in the vector list.

- **The function returns tuples, not just IDs.** The caller needs the RRF score to display in the UI (the "RRF 0.0326" badge you see on each result card).

2. How search_table orchestrates the fusion -- `search_api/services/search.py`

```
def search_table(table, query, limit=20, offset=0, ...):
    # Step 1: Embed the query into a vector
    query_vec = embed_query(query)

    conn = get_connection()
    with conn.cursor(cursor_factory=RealDictCursor) as cur:
        # Step 2: Run vector search -- returns IDs sorted by cosine similarity
        vector_rows = _vector_search(cur, table, query_vec, detected_lang, ...)
        # Step 3: Run BM25 search -- returns IDs sorted by ts_rank
        bm25_primary_rows = _bm25_search(cur, table, query, detected_lang, ...)

        # Extract just the IDs (RRF only needs rank order, not scores)
        vector_ids = [r[id_col] for r in vector_rows]
        bm25_primary_ids = [r[id_col] for r in bm25_primary_rows]

        # Step 4: Cross-language fallback (if primary BM25 results are sparse)
        bm25_secondary_ids = []
        if should_fallback(bm25_primary_dicts):
            bm25_secondary_rows = _bm25_search(cur, table, query, secondary_lang, ...)
            bm25_secondary_ids = [r[id_col] for r in bm25_secondary_rows]

        # Step 5: Fuse all three ranked lists into one
        fused = fuse_rankings(vector_ids, bm25_primary_ids, bm25_secondary_ids)
```

Notice how the raw scores (`cosine_similarity` , `bm25_rank_score`) are never passed to `fuse_rankings` . Only the ordered ID lists are passed -- RRF derives everything it needs from the position of each ID in each list.

3. Score example walkthrough

Suppose a query returns:

Source	Rank 1	Rank 2	Rank 3
Vector	cat_42	cat_17	cat_99
BM25 primary	cat_17	cat_42	cat_55

With $k=60$:

- **cat_42**: appears at vector rank 1 and BM25 rank 2
 - $1/(60+1) + 1/(60+2) = 0.01639 + 0.01613 = 0.03252$
- **cat_17**: appears at vector rank 2 and BM25 rank 1
 - $1/(60+2) + 1/(60+1) = 0.01613 + 0.01639 = 0.03252$
- **cat_99**: appears at vector rank 3 only
 - $1/(60+3) = 0.01587$
- **cat_55**: appears at BM25 rank 3 only
 - $1/(60+3) = 0.01587$

cat_42 and cat_17 tie because they both appear once at rank 1 and once at rank 2. Items appearing in both lists score roughly double compared to items in only one list -- this is RRF's core value proposition.

How to explain this in an interview

"We use Reciprocal Rank Fusion to combine vector search and BM25 keyword search into a single result ranking. RRF works purely on rank positions rather than raw scores, which solves the fundamental problem that cosine similarity (0 to 1) and BM25 `ts_rank` (unbounded positive) are on incomparable scales -- a weighted average would let whichever source produces larger numbers dominate. The formula is $1/(k+rank)$ where $k=60$ is a dampening constant from the original 2009 paper. Items that appear highly ranked in both the vector and BM25 lists accumulate score from both, naturally surfacing results that are both semantically relevant and keyword-matched. We also support a secondary-language BM25 source at half weight for cross-language fallback."